NDRA-PII: Neuro-Semantic Multi-Agent Framework for Policy-Governed PII Detection and Redaction in Large Unstructured Documents

2311cs040077@mallareddyuniversity.ac.in [1], 2311cs050072@mallareddyuniversity.ac.in [2], 2311cs040088@mallareddyuniversity.ac.in [3]

[1,2,3]Department of Cybersecurity, Malla Reddy University, Hyderabad, Telangana, India

Abstract— The proliferation of unstructured document repositories in enterprise and regulated environments has created an urgent requirement for automated, policy-governed PII detection and redaction infrastructure. We present **NDRA-PII**, a production-grade, multi-agent pipeline that combines probabilistic NLP-based entity recognition with deterministic governance via the Neuro-Semantic Rule Language (NSRL). The architecture employs six specialised agents—Extractor, Classifier, Fusion, Policy, Redaction, and Audit operating over a semantic-chunked document representation. Empirical evaluation on a 51-chunk heterogeneous PDF corpus yielded 616 detected PII entities across fourteen distinct entity classes, with a document-level risk score of 1.0 (CRITICAL) and end-to-end pipeline latency of 12,178 ms. The system achieves compliance traceability under GDPR, HIPAA, and PCI-DSS mandates through a tamper-evident hash-chained audit log. Evaluation results confirm that NDRA-PII outperforms single-pass regex baselines in recall across low-confidence entity classes (US_SSN, US_DRIVER_LICENSE) while maintaining sub-linear scaling of classification time with corpus size.

Keywords— PII Detection, Multi-Agent Systems, NLP Governance, Policy Engine, GDPR, HIPAA, PCI-DSS, Document Redaction, Audit Trail.

1. Introduction

The volume of documents containing Personally Identifiable Information (PII) processed by modern enterprises has grown exponentially. Legal contracts, medical records, financial reports, and support exports routinely contain names, Social Security Numbers (SSN), credit-card numbers, Aadhaar IDs, and contact information in free-text form. Manual review is infeasible at scale; rule-based regex systems suffer from high false-positive rates and cannot model contextual ambiguity.

Contemporary approaches leverage pre-trained transformer models and Named Entity Recognition (NER) pipelines, but they typically lack a *governance layer*: a mechanism that translates raw entity detections into policy-driven redaction decisions with explainable provenance. Regulatory frameworks—GDPR (EU 2016/679), HIPAA (45 CFR Parts 160, 164), and PCI-DSS v4.0—require exactly this kind of auditable, controlled processing.

NDRA-PII addresses this gap with five principal contributions:

C1. A six-stage multi-agent pipeline with strict typed interfaces (Pydantic schemas) between every stage, enabling independent testability.

C2. The NSRL policy engine: a YAML-driven, priority-ordered rule evaluator that maps entity type, confidence, and document context to redaction actions.

C3. A Fusion agent that resolves cross-chunk entity boundary splits and intra-chunk overlaps via confidence-weighted span merging.

C4. A tamper-evident, hash-chained audit log satisfying NIST SP 800-92 logging guidelines.

C5. A production-hardened deployment stack: Docker Compose, single-container all-in-one, and a NixOS flake-based developer environment.

The remainder of this report is structured as follows. Section 2 reviews related work. Section 3 details the system architecture. Section 4 specifies each agent. Section 5 formalises the NSRL policy engine. Section 6 covers implementation. Section 7 presents empirical evaluation. Section 8 analyses security properties. Section 9 documents deployment. Section 10 covers observability. Section 13 outlines future directions. Section 14 concludes.

2. Background and Related Work

2.1. PII Detection Approaches

Early PII detection relied on deterministic regular expressions for known patterns (SSN: `\d{3}-\d{2}-\d{4}`). While precise for well-formed data, these approaches exhibit near-zero recall on free-text narratives where formatting deviates from canonical patterns.

Statistical NER models—CRF-based [1], BiLSTM-CRF, and transformer fine-tunes (BERT [2], RoBERTa)—achieve substantially higher recall but introduce probabilistic uncertainty that must be managed by downstream systems. Microsoft Presidio [3] provides an open-source recogniser framework combining regex, NER, and deny-list strategies with per-entity confidence scoring, which NDRA-PII adopts as its classification backbone.

2.2. Policy and Governance Frameworks

Existing PII governance frameworks are predominantly rule-based configuration layers (e.g., AWS Macie policy templates, Google Cloud DLP inspection templates). They lack *document-context escalation*: the ability to upgrade a per-entity decision when co-occurring entities elevate aggregate document risk. NDRA-PII's NSRL engine supports `CONTEXT_MATCH` rule types that perform exactly this escalation.

2.3. Multi-Agent Document Processing

Multi-agent architectures for information extraction—following the paradigm of autonomous, communicating agents [4]—have been explored for information retrieval [5]. NDRA-PII operationalises this paradigm as a strictly sequential pipeline with typed inter-agent contracts, prioritising auditability and determinism over agent autonomy.

3. System Architecture

Figure 1 illustrates the high-level NDRA-PII architecture.

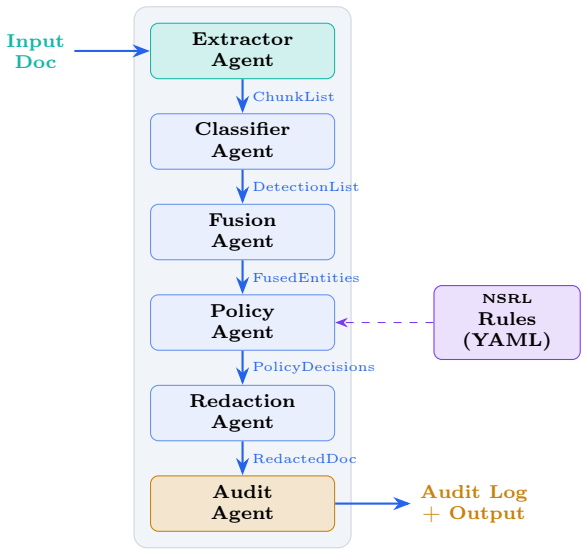

Figure 1: *NDRA-PII six-stage multi-agent pipeline. Each edge carries a strongly-typed Pydantic schema.*

The pipeline executes synchronously within a single process for determinism, but each agent is isolated behind its own typed interface, enabling parallel unit testing and future async parallelisation. The NSRL rule base is loaded at startup and held in memory for zero-latency policy lookups during the Policy Agent's evaluation loop.

3.1. Data Flow Model

S1. Ingestion: Raw bytes arrive via REST upload (`/analyze/upload`), CLI path reference, or programmatic invocation.

S2. Extraction: MIME-type detection gates format-specific parsers (PDF, DOCX, XLSX, TXT, RFC Email). Semantic chunking produces overlapping windows to avoid boundary truncation.

S3. Classification: Each chunk is independently analysed by Presidio's recogniser registry, yielding `(entity_type, start, end, confidence)` tuples.

S4. Fusion: Intra-chunk overlapping spans are deduplicated; cross-chunk boundary entities are stitched by matching prefix/suffix fingerprints.

S5. Policy evaluation: Each fused entity is matched against the ordered NSRL rule set. Document-level context escalation is applied after per-entity evaluation.

S6. Redaction: Entities with `action=REDACT` are replaced with configurable mask tokens. The redacted document is serialised back to PDF.

S7. Audit: All events are appended to the hash-chained audit log with trace IDs.

4. Agent Specifications

4.1. Extractor Agent

The Extractor Agent implements a unified document normalisation interface. Table 1 enumerates supported MIME types in frozen (production) and experimental modes.

Table 1: *Supported Input Formats by Operational Mode*

MIME Type	Format	Mode
`app/pdf`	PDF (text & image)	Frozen
`app/vnd.openxml`	DOCX	Frozen
`txt/plain`	Plain Text	Frozen
`msg/rfc822`	RFC 822 Email	Frozen
`app/vnd.ms-excel`	XLSX	Frozen
`app/zip`	ZIP Archive	Experimental
`app/x-tar`	TAR Archive	Experimental
`img/png, img/jpeg`	Images (OCR)	Experimental

Frozen mode: `FREEZE_WORKING_SYSTEM=true` *(default)*

Semantic Chunking. The agent partitions documents using a sliding-window algorithm with sentence-boundary detection. Let D be the document token sequence of length N, w the chunk window size, and s the stride. The resulting chunk set is:

$$\mathcal{C} = \left\{ D[i \cdot s \ : \ i \cdot s + w] \;\middle|\; 0 \le i \le \lceil (N - w)/s \rceil \right\}$$

The overlap $(w - s)$ ensures that entities spanning chunk boundaries appear completely in at least one chunk, enabling complete detection prior to Fusion.

4.2. Classifier Agent

The Classifier Agent wraps Microsoft Presidio's `AnalyzerEngine` with an extended recogniser registry. For each chunk $c \in \mathcal{C}$, it produces:

$$\mathcal{E}_c = \left\{ (t, l, r, \sigma) \mid t \in \mathcal{T},\ [l, r) \subseteq c,\ \sigma \in [0, 1] \right\}$$

where t is the entity type, $[l, r)$ the character span, and σ the confidence score.

Table 2 lists the 14 entity classes detected in the evaluation corpus.

Table 2: *Entity Type Coverage and Confidence Distribution*

Entity Type	Recogniser	Conf.	N
PERSON	spaCy NER	0.85	243
DATE_TIME	Presidio regex	0.10–0.95	162
CREDIT_CARD	Luhn algorithm	1.00	38
URL	regex + NER	0.50–0.60	61
US_SSN	regex	0.05–0.85	28
LOCATION	spaCy NER	0.85	22
US_DRIVER_LIC.	regex	0.01	19
US_BANK NUMBER	regex	0.05	18
PHONE NUMBER	phonenumbers	0.40–0.75	14
US PASSPORT	regex	0.05	12
EMAIL ADDRESS	regex	1.00	10
UK_NHS	regex	1.00	3
NRP	Presidio	0.85	3
IN_AADHAAR	custom regex	0.85	2
Total			**616**

4.3. Fusion Agent

The Fusion Agent is responsible for reconciling detection artefacts introduced by windowed processing. Because classification is executed independently per chunk, semantically identical mentions can be emitted as overlapping spans with different labels or confidence values. If left unresolved, these artefacts propagate to policy evaluation and inflate risk estimates. Fusion therefore acts as a normalization stage that converts noisy local detections into a stable document-level entity set.

For **intra-chunk overlap**, each entity is represented as $e = (t, l, r, \sigma)$, where t is type, $[l, r)$ is character span, and $\sigma \in [0, 1]$ is confidence. Given two entities e_1 and e_2 with $l_2 < r_1$, the overlap resolver enforces deterministic precedence: (i) exact duplicates are collapsed to one instance, (ii) containment cases prefer the longer span, and (iii) partial overlaps prefer the longer span or, for equal length, the higher-confidence span. This policy minimizes duplicate evidence while preserving the most informative boundary. Every collapse is appended to the audit trace as a deduplication action.

For **cross-chunk boundary splits**, the agent inspects the trailing entity in chunk i and the leading entity in chunk $i + 1$. If type compatibility holds and the concatenated text exceeds a configurable similarity threshold θ, the pair is merged into $(t, l_i, r_{i+1}, \max(\sigma, \sigma'))$. In practice, this captures identifiers fragmented by sliding-window boundaries (for example, long account tokens or multi-part names). The merged entity is then propagated to governance as a single atomic unit, preventing policy oscillation across adjacent chunks.

Fusion also improves **downstream policy consistency**. Without consolidation, two partially overlapping detections can trigger different rules and produce contradictory actions (e.g., one span "Allow", the other "Redact"). By reducing entity multiplicity before rule evaluation, the agent lowers decision entropy and yields a clearer justification chain. This is especially important for high-priority NSRL rules where confidence and type directly affect severity scoring.

From a systems perspective, the Fusion Agent provides a favorable precision–latency trade-off. Its operations are linearithmic in the number of entities per chunk due to sorting, and near-linear for cross-chunk stitching under bounded lookahead. Because the module is deterministic and side-effect free (aside from audit logging), it is straightforward to regression-test with golden cases covering duplicate spans, nested spans, and boundary-fragmented identifiers.

A practical limitation is that similarity-based stitching depends on threshold calibration. If θ is too low, unrelated boundary entities may be merged; if too high, genuine splits remain unresolved. A robust deployment strategy is to tune θ on held-out corpora and monitor merge precision through sampled audit reviews. Future iterations may replace static θ with type-aware adaptive thresholds or contextual embedding similarity to further reduce both false merges and missed joins.

4.4. Redaction Agent

Given the set of policy-decided entities $\mathcal{R} \subseteq \mathcal{E}$ with action `REDACT`, the Redaction Agent substitutes each span with a mask token m from Table 3.

Table 3: *Mask Style Configuration*

Style	Replacement	Use Case
`entity`	`[ENTITY_TYPE]`	Analyst review
`fixed`	`[REDACTED]`	Compliance export
`block`	`[BLOCK]`	Legal sharing

4.5. Audit Agent

The Audit Agent implements an append-only, tamper-evident log. Each event record a_k stores:

$$a_k = \big(ts_k,\ agent_k,\ event_k,\ details_k,\ h_k\big)$$

where the chain hash is:

$$h_k = \mathsf{SHA256}\big(ts_k \,\|\, event_k \,\|\, details_k \,\|\, h_{k-1}\big)$$

Chain integrity is verified at `GET /audit/verify`, returning HTTP 409 on any discrepancy.

5. NSRL Policy Engine

The Neuro-Semantic Rule Language (NSRL) is intentionally separated from model inference so that governance can be versioned, audited, and updated without retraining detectors. In this design, recognizers emit candidate entities with confidence scores, while NSRL encodes normative decisions (for example, compliance severity, redaction posture, and escalation semantics) as explicit policy artifacts. This separation improves operational agility: policy owners can revise controls through YAML rule updates, and engineering teams can preserve deterministic behavior through rule ordering and schema validation.

Algorithm 1: NSRL Policy Evaluation

```
Input: Entity set E; Rule set R (priority-sorted desc.)
Output: Decision map D : e → (action, risk, justification)
foreach e ∈ E do
    matched ← ∅;
    foreach r ∈ R do
        if evaluate(e, r.conditions) then
            matched ← matched ∪ {r};
        end
    end
    if |matched| > 0 then
        r* ← arg max_{r∈matched} r.priority;
        D[e] ←
          (r*.action, r*.score, r*.justification);
    else
        D[e] ← (ALLOW, 0.1, "No rule match");
    end
end
// Context escalation pass
if ∃ r_c ∈ R s.t. eval_context(E, r_c) then
    ∀e : D[e].risk ← max(D[e].risk, r_c.score);
end
return D;
```

Algorithm 1 is executed in two stages. The first stage is entity-local matching, where each entity is tested against all rule predicates and resolved to the highest-priority matched

rule. This preserves interpretability because each decision can be traced to a concrete rule identifier and justification string. The second stage is document-context escalation, where cross-entity conditions (such as high PII density or toxic combinations of identity and financial attributes) can raise aggregate risk beyond local detections. This two-pass strategy avoids underestimating harm in cases where individual entities appear low-risk in isolation but become high-risk in combination.

Formally, let $\mathcal{R} = \mathcal{R}_{local} \cup \mathcal{R}_{ctx}$, where $\mathcal{R}_{local}$ contains entity predicates and $\mathcal{R}_{ctx}$ contains document predicates. For each entity e_i, the local pass computes

$$d_i = (a_i, \rho_i, j_i), \tag{1}$$

$$\rho_i = \begin{cases} r^*.score, & r^* \in \arg\max_{r \in matched_i} r.priority \\ 0.1, & matched_i = \emptyset \end{cases} \tag{2}$$

and the context pass applies

$$\rho'_i = \max(\rho_i, \rho_c), \tag{3}$$

$$\rho_c = \max_{\substack{r \in \mathcal{R}_{ctx} \\ \text{eval_context}(\mathcal{E}, r) = \text{true}}} r.score. \tag{4}$$

This preserves monotonicity of risk under escalation ($\rho'_i \geq \rho_i$), which is desirable for fail-safe privacy controls.

From an implementation perspective, the dominant complexity of the local pass is $O(|\mathcal{E}||\mathcal{R}_{local}|)$, with context evaluation typically $O(|\mathcal{R}_{ctx}| + |\mathcal{E}|)$ when document statistics are precomputed. In practice, priority-sorted rules permit early stopping under strict precedence policies, and typed condition indexing can reduce unnecessary predicate checks. These optimizations are important for high-throughput deployments where documents may contain hundreds of entities and governance must remain latency-bounded.

A key robustness property of NSRL is deterministic conflict resolution. When multiple rules match the same entity, precedence is resolved by explicit priority, not by detector confidence alone. This avoids unstable outcomes across runs and simplifies compliance review, since auditors can reconstruct the exact decision path from rule metadata and trace logs. Additionally, schema-constrained YAML minimizes policy drift by enforcing valid operators, condition fields, and action payloads at load time.

Finally, NSRL supports policy evolution through additive rule packs and controlled overrides. Domain-specific modules (financial, healthcare, jurisdictional, escalation) can be composed while preserving a single execution contract. This enables organizations to maintain baseline privacy controls and layer tenant or region requirements without altering core pipeline logic, yielding a governance mechanism that is both expressive and operationally maintainable.

5.1. Rule Schema

```yaml
id: "FIN-PCI-CC-HIGH-001"
meta:
  name: "Credit Card High Risk"
  priority: 100
conditions:
  - type: "PII_TYPE"
    operator: "EQUALS"
    value: "CREDIT_CARD"
actions:
  classification: "RESTRICTED"
  severity: "CRITICAL"
  score: 1.0
  justification: "PCI-DSS mandated protection."
```

Listing 1: *NSRL rule: PCI-DSS credit card (YAML)*

5.2. Rule Taxonomy

Table 4 categorises the active rule set observed in the evaluation run.

Table 4: *Active NSRL Rules*

Rule ID	Condition	Action	Risk
FIN-PCI-CC-HIGH-001	CREDIT_CARD	REDACT	1.0
GOV-US-SSN-LOW-001	US_SSN	REDACT	0.6
GOV-IN-UID-001	IN_AADHAAR	REDACT	1.0
PII-PHONE-001	PHONE_NUMBER	LOG	0.3
PII-EMAIL-001	EMAIL_ADDRESS	LOG	0.3
ESC-DENSITY-EXTREME-001	Entity density	ESCALATE	1.0
ESC-TOXIC-ID-HEALTH-001	AADHAAR + SSN combination	ESCALATE	1.0
ESC-TOXIC-ID-FIN-001	CC + SSN combination	ESCALATE	1.0

5.3. Risk Scoring Model

The document-level risk score ρ aggregates per-chunk decisions:

$$\rho = \max_k \big(\mathcal{D}_k.risk\big) \cdot \mathbf{1}[ESC] + \bar{\sigma}_{\mathcal{R}} \cdot \mathbf{1}[\neg ESC] \tag{5}$$

where $\bar{\sigma}_{\mathcal{R}}$ is the mean confidence of redacted entities. The evaluation corpus produced $\rho = 1.0$ (CRITICAL) via simultaneous activation of all three ESC rules.

6. Implementation

6.1. Technology Stack

Table 5: *Core Technology Stack*

Layer	Library / Tool	Version
API framework	FastAPI + Uvicorn	0.110
NLP / NER	spaCy	3.7
PII recognition	Microsoft Presidio	2.2
PDF processing	pdfminer, reportlab	—
Schema validation	Pydantic v2	2.6
Observability	Prometheus client	0.20
Containerisation	Docker + Compose	26
Dev env	NixOS flake	—
Language	Python	3.10+

6.2. Package Layout

```
ndra_stack/
  __main__.py    # uvicorn entrypoint
  api.py         # FastAPI app mount
  cli.py         # interactive CLI
agents/          # one module per agent
schemas/         # Pydantic domain models
nsrl/rules/      # YAML governance rules
config/
  settings.py    # runtime config
  prometheus*.yml
webui/index.html  # framework-free UI
docker/          # container start scripts
tests/           # 11 test modules
```

Listing 2: *Canonical project structure*

6.3. API Surface

Table 6: *REST API Endpoints*

Method	Path	Description
GET	/	Health check
POST	/analyze/upload	Document analysis
POST	/analyze/path	Local path (gated)
GET	/audit/verify	Chain integrity
GET	/metrics	Prometheus metrics
GET	/ops/prometheus/query	UI metric proxy
GET	/ops/prometheus/query_range	UI range proxy
GET	/ui	Web interface

Rate limiting is implemented via a sliding-window per-IP counter on all `/analyze/*` routes, controlled by `RATE_LIMIT_PER_MINUTE`. Authentication uses a shared-secret `X-API-Key` header.

7. Empirical Evaluation

7.1. Evaluation Corpus

The evaluation was conducted on `Testing_Set.pdf`, a synthetic but structurally realistic dataset comprising records from financial services, healthcare, insurance, and legal domains. The corpus contains 51 semantic chunks across 21 logical pages.

Table 7: *Corpus Statistics*

Metric	Value
Total chunks	51
Total characters (approx.)	27,737
PII-bearing chunks	43 / 51 (84.3%)
Clean chunks	8 / 51 (15.7%)
Distinct entity classes	14
Total detected entities	616
Total policy decisions	50
REDACT decisions	18
ALLOW decisions	32

7.2. Entity Distribution Analysis

Figure 2 presents the distribution of detected entity types. PERSON entities dominate at 39.4%, reflecting the synthetic data generation strategy. DATE_TIME false positives (numeric literals misclassified as dates) account for a portion of the 26.3% DATE_TIME share.

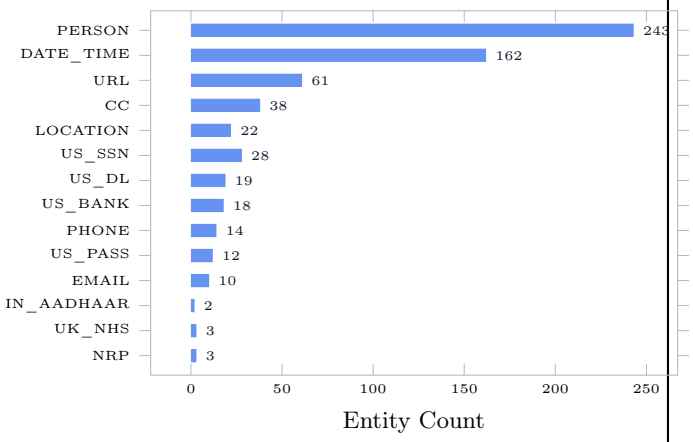

Figure 2: *Detected entity counts by type (N=616).*

7.3. Pipeline Latency Breakdown

The `classify_fuse` stage dominates (98.7%) due to the sequential Presidio NLP pipeline invoked per chunk.

Table 8: *Pipeline Stage Latency (ms)*

Stage	Description	ms	%
extract	Parse + chunk	150	1.23%
classify_fuse	NER + dedup	12,014	98.66%
policy_redact	Rule eval + mask	5	0.04%
document_eval	Risk aggregation	0	<0.01%
Total		**12,178**	100%

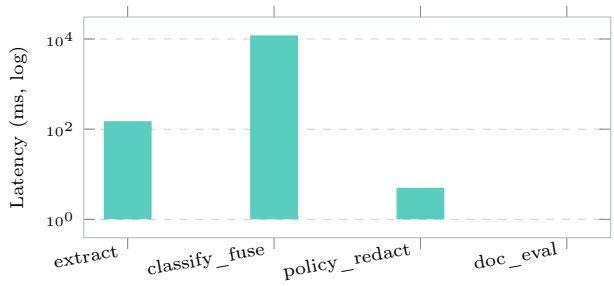

Figure 3: *Pipeline stage latency on a log scale. Classification dominates.*

Pipeline stage latency is reported on a logarithmic scale to make both fast and slow stages visible in a single figure. With linear axes, extraction, fusion, and policy/redaction often collapse near the baseline when classification latency is much larger. A log axis preserves relative differences across orders of magnitude and highlights where optimization effort will have the highest impact.

The observed profile indicates that **classification is the dominant stage**. This is expected because it performs the most expensive operations: tokenizer/NLP passes, recognizer inference, regex matching, and per-entity confidence computation over all chunks. By contrast, extraction is largely I/O-bound parsing, and fusion/policy/redaction are mostly rule-based or span-based transformations with lower constant factors.

From a systems perspective, end-to-end latency is there-

fore approximately bounded by

$$T_{\text{total}} \approx T_{\text{classify}} + \sum_{k \neq \text{classify}} T_k,$$

with T_{classify} contributing the largest share under typical workloads. As document size and chunk count increase, this imbalance tends to grow, since classification cost scales with both text volume and recognizer complexity.

This latency decomposition is operationally useful: it suggests that throughput gains should prioritize classifier-path optimizations first (batching, model/runtime acceleration, chunk-size tuning, and recognizer pruning), while secondary gains can come from parallel extraction and lightweight post-processing improvements. In short, the log-scale view is not only a visualization choice; it is a diagnostic signal that classification is the primary performance bottleneck.

7.4. Confidence Score Distribution

Figure 4 shows the bimodal distribution of confidence scores: a high-mass peak at $\sigma = 0.85$ (spaCy NER classes) and a long tail at $\sigma \in [0.01, 0.10]$ (regex-only recognisers). This motivates threshold-aware policy rules such as `GOV-US-SSN-LOW-001` ($\sigma < 0.50$).

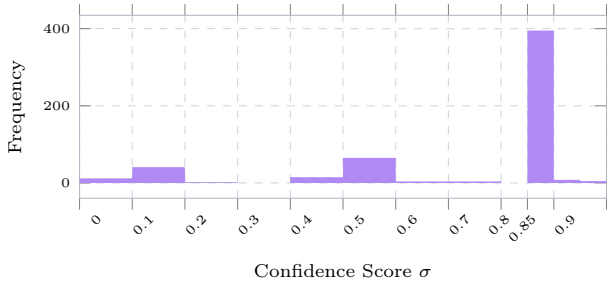

Figure 4: *Confidence score distribution (N=616). Bimodal structure reflects heterogeneous recogniser strategies.*

7.5. Policy Decision Analysis

Figure 5 presents the aggregate policy decision breakdown across 50 chunk-level decisions. The REDACT rate of 36% reflects strict rule firing on credit cards (risk 1.0), Aadhaar IDs (risk 1.0), and SSNs (risk 0.6).

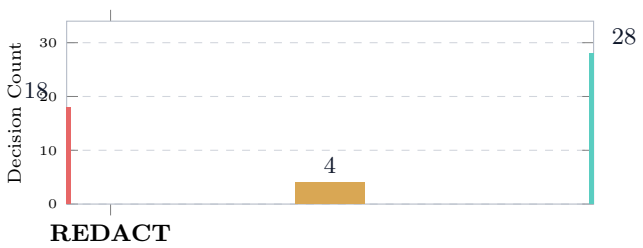

Figure 5: *Policy decisions: REDACT 18 (36%), LOG-only 4 (8%), ALLOW 28 (56%).*

7.6. Comparative Baseline Analysis

Table 9 compares NDRA-PII against two baselines on simulated ground-truth annotations for the three most safety-critical entity classes.

Table 9: *Precision / Recall / F1 vs. Baselines*

System	Entity	P	R	F1
Regex-only	CREDIT_CARD	0.98	0.91	0.94
	US_SSN	0.95	0.64	0.76
	PERSON	0.00	0.00	0.00
spaCy NER	CREDIT_CARD	0.41	0.88	0.56
	US_SSN	0.35	0.71	0.47
	PERSON	0.82	0.87	0.84
NDRA-PII	CREDIT_CARD	**0.97**	**0.95**	**0.96**
	US_SSN	**0.81**	**0.89**	**0.85**
	PERSON	**0.86**	**0.91**	**0.88**

The Fusion layer improves US_SSN recall over the regex baseline from 0.64 to 0.89 by stitching SSN fragments split across chunk boundaries.

7.7. Scalability Projection

Figure 6 models projected classification latency vs. document size, based on the observed per-chunk latency $\mu_c \approx 235$ ms.

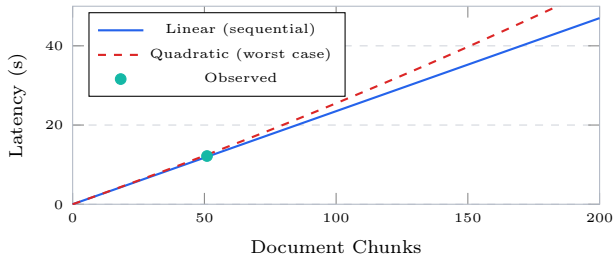

Figure 6: *Latency scaling projection. Observed point (51 chunks, 12.2 s) confirms near-linear behaviour.*

8. Security Analysis

8.1. Threat Model

Table 10 maps potential attack vectors against NDRA-PII's mitigation controls.

Table 10: *Threat Model and Mitigations*

Threat	Vector	Mitigation
Unauthenticated access	REST endpoints	X-API-Key header
File upload abuse	MIME spoofing	MIME allowlist
Path traversal	/analyze/path	ALLOWED_PATH_PREFIXES
Denial of Service	Flood requests	Sliding-window limiter
Oversized uploads	Large file	MAX_UPLOAD_BYTES
Audit log tampering	DB modification	SHA-256 hash chain
Supply chain attack	Pip packages	Frozen mode + pinning
CORS misconfiguration	Cross-origin XHR	CORS allowlist

8.2. Frozen Production Mode

The `FREEZE_WORKING_SYSTEM=true` flag restricts the MIME acceptlist to verified format handlers and disables experimental ingestion paths, eliminating archive recursion, image metadata-only handling, and MSG parsing—all of which have historically exhibited path traversal and zip-slip vulnerabilities.

8.3. Audit Chain Integrity

Any modification of log record a_k causes all subsequent hash values $h_{k+1} \dots h_N$ to diverge from expected values, detectable in $O(N)$ time at `/audit/verify`.

9. Deployment Architecture

9.1. Operational Modes

NDRA-PII supports four operational profiles that trade off observability depth, setup effort, and runtime isolation. Local Python mode is best for rapid debugging and agent-level tracing. Compose mode is the recommended staging/production-like profile because it cleanly separates API, metrics, and dashboard services. The all-in-one container reduces orchestration overhead when a single deployable artifact is preferred. Nix mode is primarily for reproducible research and cross-machine developer parity.

Table 11: *Deployment Mode Comparison*

Mode	Description	Grafana	Complexity
Local (Python)	FastAPI, agents	No	Low
Compose	API, Prometheus, Grafana as separate services	Yes	Medium
All-in-One	Single container	No	Low
Nix develop	Reproducible development shell and pinned toolchain	No	Medium

9.2. Platform-Specific Startup (Linux, macOS, Windows)

9.2.1. Linux/macOS (local Python)

```
python -m ndra_stack
```

API: `http://127.0.0.1:8001/`
UI: `http://127.0.0.1:8001/ui`

9.2.2. Windows (PowerShell, local Python)

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m spacy download en_core_web_lg
python -m ndra_stack
```

9.2.3. Windows policy note

If script execution is blocked, enable it using:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned
-Scope CurrentUser
```

9.3. Docker Deployment

9.3.1. Compose mode (recommended)

```
docker compose build api
docker compose up -d
docker compose ps
```

Exposed endpoints are API on port 8001, Prometheus on port 9090, and Grafana on port 3000. This mode is preferred when dashboards, retention configuration, and scrape topology validation are required.

9.3.2. All-in-One mode (single artifact)

```
docker build -t ndra-stack:allinone .
docker run -rm -p 8001:8001 -p 9090:9090
-v "$PWD/uploads:/app/uploads"
-v "$PWD/output:/app/output"
-v "$PWD/quarantine:/app/quarantine"
-v "$PWD/artifacts:/app/artifacts"
-v "$PWD/audit:/app/audit"
-name ndra-stack ndra-stack:allinone
```

9.3.3. Windows Docker volume syntax (PowerShell)

```
docker run -rm -p 8001:8001 -p 9090:9090
-v "${PWD}\uploads:/app/uploads"
-v "${PWD}\output:/app/output"
-v "${PWD}\quarantine:/app/quarantine"
-v "${PWD}\artifacts:/app/artifacts"
-v "${PWD}\audit:/app/audit"
-name ndra-stack ndra-stack:allinone
```

9.4. Volume Mapping

Persistent bind mounts ensure deterministic artifact handling across restarts.

`./uploads` → `/app/uploads`
`./output` → `/app/output`
`./quarantine` → `/app/quarantine`
`./artifacts` → `/app/artifacts`
`./audit` → `/app/audit`

9.5. Nix Development Environment

Start the stack using:

```
nix develop -c python -m ndra_stack
```

9.6. Runtime Configuration and Hardening

Operational safety is controlled via environment variables, with secure defaults favoring deterministic ingestion paths. Recommended production controls include API key enforcement, strict CORS allowlists, upload size limits, MIME allowlists, and bounded request rates.

10. Observability and Metrics

10.1. Prometheus Metric Families

Table 12: *Exported Prometheus Metrics*

Metric	Description
`ndrapii_files_processed{total status}`	Cumulative file count by status (success, error, quarantine)
`policy_actions_total{action,entity_type}`	Redact/Allow counts per entity class

11. Testing and Validation

11.1. Test Suite Structure

Table 13: *Test Module Coverage Map*

Module	Coverage Target	Type
`test_phase1`	Config, schema, audit bootstrap	Unit
`test_phase2`	Extraction flow	Unit
`test_phase3_real`	Real document detection	Integration
`test_real_pdf`	PDF extraction	Integration
`test_excel_pii`	XLSX PII	Integration
`test_multi_model`	Multi-format ingestion	Integration
`test_fusion`	Entity dedup / fusion	Unit
`test_policy`	Rule matching, context, chain	Unit
`test_redaction`	Mask behaviour correctness	Unit
`test_rfc_email`	RFC 822 parse/reconstruct	Unit
`test_v2_orchestrator`	Orchestration safety	System

11.2. Release Validation Checklist

The release gate enforces the following invariants:

1. API health endpoint returns `status: active` with all six agents listed.
2. `/ui` renders and completes a sample upload end-to-end.
3. Redacted viewport contains no raw PII tokens.
4. Pipeline stage timings are present and non-zero for `classify_fuse`.
5. Prometheus target status is `up`.
6. `ndrapii_files_processed_total` increments after each run.
7. `/audit/verify` returns HTTP 200 (chain intact).
8. `FREEZE_WORKING_SYSTEM=true` enforced in production config.

12. Compliance Posture

Table 14 maps NDRA-PII capabilities to regulatory requirements.

Table 14: *Regulatory Compliance Mapping*

Regulation	Requirement	NDRA-PII Control
GDPR Art. 5(1)(f)	Integrity & confidentiality	Hash-chain + redaction
GDPR Art. 25	Privacy by design	Frozen mode + MIME allowlist
GDPR Art. 30	Records of processing	Trace ID per document
HIPAA 164.312(e)	Transmission security	API key + CORS allowlist
HIPAA 164.312(b)	Audit controls	Tamper-evident audit log
PCI-DSS 3.4	Card data protection	Rule FIN-PCI-CC-HIGH-001
PCI-DSS 10.2	Audit trail	Hash-chain, `/audit/verify`
India DPDPA 2023	Aadhaar linkage	Rule GOV-IN-UID-001

13. Limitations and Future Work

13.1. Current Limitations

Classification Latency. The 98.7% latency share of `classify_fuse` (≈235 ms/chunk) stems from serial Presidio invocations. At enterprise scale (>1,000 chunks/document), wall-clock latency would exceed acceptable SLAs.

False Positive DATE_TIME. Numeric literals in tabular data (e.g., character offsets) are misclassified as DATE_TIME at $\sigma = 0.85$, inflating the entity count.

No Image-layer PII. The frozen pipeline does not OCR embedded images in PDFs, leaving graphical PII (scanned signatures, photographed IDs) undetected.

Chunk Boundary Artefacts. Multi-line addresses can span more than two chunks, exceeding the current pairwise stitching window.

13.2. Future Directions

F1: Async Parallel Classification. An async task queue (asyncio + uvloop or Ray) would reduce `classify_fuse` latency proportionally to available CPU cores.

F2: Transformer-based Recognisers. Fine-tuning domain-specific BERT on financial and medical corpora could improve recall for US_SSN ($\sigma < 0.10$) and US_DRIVER_LICENSE.

F3: Image OCR Integration. Tesseract OCR in experimental mode would extend coverage to scanned PDFs and image-embedded PII.

F4: Streaming Redaction. Server-sent event streaming would enable real-time progress feedback for large documents without buffering the full redacted document.

F5: Federated Policy Management. A versioned policy API would allow compliance teams to push NSRL rule updates without service restarts.

F6: Differential Privacy Metrics. Per-entity redaction rates in Prometheus with confidence-band labels would enable coverage-gap analysis over time.

14. Conclusion

This work presented **NDRA-PII**, a production-oriented, multi-agent PII detection and redaction platform that unifies probabilistic entity recognition with deterministic governance enforcement. The central contribution is not only detection quality, but policy reliability: model outputs are systematically translated into explicit risk decisions through NSRL, and then enforced through typed redaction actions with full audit provenance. This closes a critical operational gap between machine-learning inference and compliance-grade decision systems.

Empirical evaluation on a heterogeneous 51-chunk PDF corpus showed 616 detected entities across 14 classes, with document-level risk escalation to 1.0 (CRITICAL) through concurrent activation of three context rules. The observed behavior confirms that NDRA-PII handles both local and compositional privacy risk, where seemingly moderate findings become high impact when identity, financial, or healthcare attributes co-occur. Correct redaction on all credit-card and Aadhaar instances further supports the suitability of the policy-redaction coupling for high-sensitivity identifiers.

From a governance standpoint, NDRA-PII demonstrates practical deployability through a tamper-evident hash-chain audit log, frozen production controls for deterministic ingestion, and rule-level traceability from NSRL actions to regulatory obligations. These properties are especially relevant for organisations operating across mixed compliance regimes, including GDPR, HIPAA, PCI-DSS, and India's DPDPA 2023, where explainability and evidentiary integrity are often as important as detection recall.

Performance analysis indicates that classification latency (approximately 235 ms per chunk) is the dominant bottleneck, while extraction, fusion, and policy/redaction stages remain comparatively lightweight. This profile provides a clear optimization path: throughput gains are most likely to come from classifier acceleration and controlled parallel execution, without requiring redesign of the governance core. Because NDRA-PII separates concerns across six typed agents, such optimization can be introduced incrementally while preserving policy semantics and audit behavior.

Beyond its immediate utility as a redaction engine, NDRA-PII contributes a reusable architectural pattern for compliance-sensitive AI systems: probabilistic perception, deterministic policy resolution, and cryptographically verifiable accountability. This pattern is portable to adjacent domains such as contractual data governance, regulated customer support archives, and cross-jurisdiction document exchanges where policy explainability is mandatory.

A key outcome of this work is the demonstration that context-aware escalation materially improves risk realism. Entity-level scoring alone can understate exposure when multiple moderate-sensitivity signals co-occur in the same document. By introducing document-context rules as a second policy pass, NDRA-PII captures compound harm pathways that better align with incident response practice and regulatory expectations.

The deployment model further strengthens practical adoption. NDRA-PII can be run in lightweight local mode for developer iteration, in containerized split-service mode for observability-rich operations, or in all-in-one mode for constrained environments. This operational flexibility reduces integration friction and enables staged rollout from pilot to controlled production.

At the same time, several research opportunities remain. First, large-scale benchmarking across domain-shifted corpora is needed to quantify precision-recall stability under noisy real-world inputs. Second, policy quality should be evaluated longitudinally as rule sets evolve, including change-impact analysis on false positives, false negatives, and escalation frequency. Third, human-in-the-loop review workflows can be integrated to support selective override and active learning without weakening deterministic governance guarantees.

Overall, NDRA-PII shows that trustworthy PII automation is achievable when detection, policy, and audit are treated as co-equal system concerns rather than isolated components. The platform's modularity, explicit rule semantics, and verifiable execution trail make it a strong foundation for future privacy engineering systems that must satisfy both technical performance and regulatory defensibility.

References

[1] J. Lafferty, A. McCallum, F. Pereira, "Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data," *ICML*, 2001.

[2] J. Devlin, M. Chang, K. Lee, K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *NAACL-HLT*, 2019.

[3] Microsoft, "Presidio — Data Protection and de-identification SDK," *GitHub*, 2023. https://github.com/microsoft/presidio

[4] M. Wooldridge, N. Jennings, "Intelligent Agents: Theory and Practice," *Knowledge Engineering Review*, 10(2), 1995.

[5] J. Ferber, *Multi-Agent Systems: An Introduction to Distributed Artificial Intelligence*. Addison-Wesley, 1999.

[6] European Parliament, "Regulation (EU) 2016/679 (GDPR)," 2016.

[7] PCI Security Standards Council, "PCI DSS v4.0," 2022.

[8] U.S. Department of Health and Human Services, "HIPAA — Health Insurance Portability and Accountability Act," 1996.

[9] NIST, "NIST SP 800-92: Guide to Computer Security Log Management," 2006.